АНАЛИТИЧЕСКИЙ ОТЧЁТ · РЕВЕРС-ИНЖИНИРИНГ AI-МОНТАЖА

Монтажная Кухня

Четыре killer-фичи вирусного монтажа — караоке-субтитры, авто-центрирование спикера, цитаты с авто-зумом и вирусные заголовки — реверс-инжинирятся из Opus Clip, Submagic, CapCut и Descript и адаптируются под движок Sereban-glitch-video-montage: с готовыми filtergraph-рецептами FFmpeg 7.1 и word-timestamps Whisper.

Killer-фичи вирусного AI-монтажа 2026

От анализа конкурентов к реализации: FFmpeg filtergraph · Whisper word timestamps · оценка виральности

ОКТЯБРЬ 2026 · ВЕРСИЯ 1.0 · ВНУТРЕННИЙ ДОКУМЕНТ

Содержание

1. Резюме для руководителя 2

1.1. Сводная таблица приоритетов 2

2. Ландшафт инструментов 2026: что реверс-инжиниринг 3

3. КФ-1. Караоке-субтитры с подсветкой активного слова 4

4. КФ-2. Авто-центрирование спикера (Active Speaker Crop) 6

5. КФ-3. Цитаты, авто-зум и звуковые акценты 7

6. КФ-4. Вирусный хук: заголовки и скоринг виральности 8

7. Дорожная карта внедрения в video-montage 9

1. Резюме для руководителя

Настоящий отчёт подводит итог разведки рынка вирусного AI-монтажа и формулирует четыре killer-фичи, которые стоит перенести в движок Sereban-glitch-video-montage методом реверс-инжиниринга. Мы проанализировали продуктовые механики лидеров категории — Opus Clip, Submagic, CapCut и Descript — и отобрали только те решения, которые реализуемы на нашем стеке (FFmpeg 7.1 + Faster-Whisper) без перехода на тяжёлые нейросетевые пайплайны и без выхода за бюджет ОЗУ воркера в 12 ГБ. Каждой фиче посвящён отдельный раздел с конкретным рецептом реализации: filtergraph-цепочками FFmpeg, схемой генерации ASS-субтитров из word-таймстемпов и структурой LLM-обработки транскрипта.

Выбор фичей сделан по трём критериям. Во-первых, доказанное влияние на удержание и CTR: все четыре механики прямо заявлены в маркетинге лидеров как драйверы retention (+40% средних просмотров у Submagic, рост удержания от караоке-подсветки слов). Во-вторых, совместимость с нашим контентом: судебные заседания, интервью и press-конференции — это dense-speech видео, где караоке-субтитры, цитаты и центрирование спикера работают лучше всего. В-третьих, стоимость внедрения: каждая фича оценивается в днях, а не неделях, и вписывается в существующий конвейер render_short.py / render_segments.

+40%
средний прирост просмотров — заявка Submagic по клипам с анимированными субтитрами

21–34 с
sweet-spot длительность Shorts по практике Opus Clip и Submagic

~40%
клипов Opus Clip отбрасывают создатели — значит, нужен ручной контроль и скоринг

1.1. Сводная таблица приоритетов

Фича	Влияние на виральность	Трудозатраты	Пик ОЗУ	Приоритет
КФ-1. Караоке-субтитры (подсветка активного слова)	Высокое: retention до конца клипа	2–3 дня	+0 ГБ (libass)	P0 — первым
КФ-4. Вирусный хук, заголовки и скоринг	Высокое: CTR обложки и первые 3 с	1–2 дня	+0.2 ГБ (LLM API)	P1
КФ-3. Цитаты, авто-зум, звуковые акценты	Среднее: динамика кадра	3–4 дня	+0.1 ГБ	P2
КФ-2. Авто-центрирование спикера	Высокое для 16:9-источников	4–6 дней	+0.2 ГБ (MediaPipe)	P3

Таблица 1. Порядок внедрения killer-фичей в video-montage

Ключевой вывод: начинать следует с караоке-субтитров (КФ-1), поскольку они дают максимальное отношение эффекта к затратам и полностью ложатся на существующую стадию субтитрирования конвейера. Второй шаг — LLM-слой (КФ-4), который не трогает видеотракт вовсе и потому почти безрисков. Авто-зум с цитатами (КФ-3) и спикер-центрирование (КФ-2) требуют более глубоких изменений в render-стадии и выполняются во вторую очередь. Детальная дорожная карта с неделями и рисками — в главе 7.

2. Ландшафт инструментов 2026: что реверс-инжиниринг

Рынок AI-монтажа коротких видео в 2026 году разделился на два лагеря. Первый — «нарезатели» длинного контента: Opus Clip (10+ млн пользователей, $215 млн оценка, инвестор SoftBank) и встроенные Magic Clips у Submagic; их ядро — поиск сильных моментов в часовой подкаст-записи, авто-рефрейм в 9:16 и скоринг виральности. Второй лагерь — «финишеры» короткого клипа: Submagic ($12/мес, 4 млн бизнесов) и шаблонная экосистема CapCut; их ядро — анимированные караоке-субтитры, авто-зумы, b-roll, звуковые эффекты и моментальный вывод под TikTok/Reels/Shorts. Descript стоит особняком: это текстовый видеоредактор, у которого мы берём механику работы с транскриптом — удаление слов-паразитов и монтаж текстом.

Наша ниша — украинские судебные заседания, интервью и press-конференции — ближе всего к первому лагерю по источнику (длинная запись выступления) и ко второму по результату (вертикальный клип с плотными субтитрами). Поэтому стратегия реверс-инжиниринга гибридная: у Opus Clip берём выбор момента, рефрейминг и скоринг; у Submagic и CapCut — визуальную упаковку (караоке, зумы, плашки); у Descript — очистку речи от паразитов и лишних пауз, что для суда критично: судья говорит медленно, и без вычистки «э-э» и длинных пауз клип не удержит зрителя. Наш движок уже умеет вырезать паузы по whisper-таймкодам (audit_gaps) — это прямая аналогия Descript «Remove Silences», и её стоит усилить.

Инструмент	Ниша 2026	Killer-механики	Что берём в video-montage
Opus Clip	Нарезка длинного видео, $15–29/мес	Virality Score; Active Speaker Detection и рефрейм 9:16; AI B-roll; кластеризация хуков	Скоринг клипов (гл. 6); авто-центрирование спикера (гл. 4); холодный старт с кульминации
Submagic	Финиш шортсов, ~$12/мес	Караоке-субтитры 123 языков; авто-зум (fast/crash/smooth); Magic B-roll; SFX и эмодзи на ключевых словах	Караоке-подсветка слова (гл. 3); авто-зум и цитатные плашки (гл. 5); ключевые слова цветом
CapCut	Массовый мобильный монтаж, free	Шаблоны velocity-монтажа; авто-каптионы с подсветкой; сцены-переходы; стили текста	Стилистика плашек и тени текста; ритм смены кадра каждые 3–5 с

Инструмент	Ниша 2026	Killer-механики	Что берём в video-montage
Descript	Текстовый редактор видео	Монтаж транскриптом; Remove filler words; Studio Sound; уплотнение пауз	Усиление вырезки пауз (уже есть audit_gaps); чистка слов-паразитов по whisper-словам

Таблица 2. Что именно реверс-инжиниринг у каждого лидера рынка

Чего сознательно не копируем: генеративный b-roll (стоковые вставки чужих съёмок в судебный контент недопустимы по этике и праву), AI-аватары, «эмоциональные» эмодзи на словах в официальных сюжетах, и full-автопилот публикации. Вместо этого фокус на четырёх фичах ниже — они усиливают наш контент, не ломая документальную достоверность, и каждое решение остаётся объяснимым: зритель видит ровно то, что было сказано на заседании, только упаковано по законам коротких видео.

3. КФ-1. Караоке-субтитры с подсветкой активного слова

Это фича №1 у Submagic и CapCut и самый быстрый способ поднять удержание: глаз зрителя «приклеивается» к подсвеченному слову и досматривает клип до конца. Механика: субтитры отображаются короткими группами из 1–3 слов (WordGroup), активное слово заливается акцентным цветом ровно на время его звучания, остальные слова полупрозрачные. Для украинской речи с её длинными словами рекомендуются группы по 2–3 слова и крупный шрифт Montserrat ExtraBold 72–84 px в кадре 1080×1920.

Технически всё уже есть в стеке: Faster-Whisper при whisper.word_timestamps=True возвращает список слов с start/end; из них собирается ASS-файл, где каждому слову соответствует karaoke-тег \k с длительностью в сотых секунды; рендер выполняет libass через фильтр ass=subs.ass — это работает на CPU и добавляет 2–5% времени кодирования. Ниже — полный рецепт: генерация ASS из word-таймстемпов и команда рендера. Стиль задаётся строкой Style: primary-цвет (#FFFFFF) используется до подсветки, karaoke-заливка SecondaryColour — акцентный жёлтый #FFD400.

Генерация караоке-ASS из word-timestamps (Python)

```
# worker/features/karaoke_subs.py — ASS из word-timestamps Faster-Whisper
from faster_whisper import WhisperModel

def build_karaoke_ass(video, out_ass='subs.ass'):
    model = WhisperModel('large-v3-turbo', device='cuda', compute_type='float16')
    words = []
    for seg in model.transcribe(video, word_timestamps=True)[0]:
        for w in seg.words:                      # {'word':'СЬОГОДНІ','start':1.2,'end':1.6}
            words.append(w)

    def chunks(ws, n=3):                         # группы по 2-3 слова
        for i in range(0, len(ws), n):
            yield ws[i:i+n]

    ev, idx = [], 0
    for g in chunks(words):
        t0 = g[0]['start']; parts = []
        for w in g:
            k = max(1, round((w['end']-w['start'])*100))   # сотые секунды
            parts.append('{\\k%d}%s' % (k, w['word'].strip()))
        end = max(0, round(t0, 2)); t0f = ts(end); t1f = ts(g[-1]['end'])
        ev.append(f'Dialogue: 0,{t0f},{t1f},Kar,,0,0,0,,'
                  + '{\\fad(60,60)}' + ' '.join(parts))
    write_ass(out_ass, ev)   # шапка [Script Info] + Style: Kar,Montserrat,76

def ts(sec):  # 00:00:01.20
    m, s = divmod(sec, 60); h, m = divmod(m, 60)
    return f'{int(h):d}:{int(m):02d}:{s:05.2f}'
```

FFmpeg: прожиг караоке-субтитров

```
# Рендер: libass поверх видео. Двуязычный режим = два Dialogue-слоя:
# верхний Kar (язык речи) + нижний Translate (перевод) со сдвигом MarginV.
ffmpeg -i cut.mp4 -vf "ass=subs.ass,format=yuv420p" \
  -c:v libx264 -preset medium -crf 19 -c:a copy out_short.mp4

# Вариант с плавной заливкой слова (karaoke fill слева направо):
# в ASS использовать тег \kf вместо \k — заливка 'пылесосом' по времени слова.
```

Существующие параметры задачи ложатся сюда без изменений БД: bilingual уже включает второй слой (русский перевод мельче и ниже), subtitle_lang выбирает язык основного слоя, а палитру подсветки можно отдать пользователю в Mini App как «Цвет акцента субтитров» (жёлтый/белый/зелёный) — один hex в params. Ожидаемый эффект по рынку: Submagic называет анимированные каптионы главным драйвером +40% просмотров; в нашем судебном контенте дополнительно решается проблема шумного зала — слова читаются даже без звука.

4. КФ-2. Авто-центрирование спикера (Active Speaker Crop)

Opus Clip называет это Active Speaker Detection: горизонтальный исходник превращается в вертикальный клип, где кадр следует за говорящим, а не режется по центру. Для судебных записей это критично: камера показывает весь зал, говорящий постоянно смещается (судья, прокурор, защитник), и центральный кроп оставляет пустую половину кадра. Механика двухступенчатая: определяем, ГДЕ лица в кадре (детекция), и КТО говорит (сегменты whisper по спикерам), затем строим траекторию окна кропа и прожигаем её в FFmpeg.

Детекция: MediaPipe Face Detection (BlazeFace) — CPU-режим, вес модели 1–2 МБ, пик ОЗУ ~150 МБ, 25–40 fps на одном ядре. Прогоняем каждый 5-й кадр, собираем центры лиц, сглаживаем экспоненциальным средним (EMA α=0.15) и квантуем траекторию до опорных точек каждые 0.5 с — они уходят в sendcmd-файл. Кто говорит: whisper-сегменты уже упорядочены по времени; сопоставляя активного лица на帧 с говорящим сегментом (совпадение активности рта опционально), выбираем целевое лицо. Результат — файл команд для фильтра sendcmd, который управляет x-координатой кропа:

Тракт авто-рефрейминга: MediaPipe + sendcmd + crop

```
# 1) Python: медиапайп -> сглаженная траектория окна кропа (x каждые 0.5 с)
# 2) FFmpeg: плавное следование кадра за спикером (1080x1920 из 1920x1080)
ffmpeg -i source.mp4 -f lavfi -t 600 -i "color=black" \
  -filter_complex \
  "[0:v]sendcmd=f=crop_track.cmd, \
   crop=608:1080:(x_expr):0, \          # окно 9:16 из 1080p
   scale=1080:1920:flags=lanczos[v]" \
  -map "[v]" -map 0:a -c:v libx264 -crf 19 -preset medium vertical.mp4

# crop_track.cmd (генерируется Python, EMA-сглаживание):
# 0.0 CROP_X 960;   0.5 CROP_X 1042;  1.0 CROP_X 1042;  1.5 CROP_X 890; ...
# FFmpeg сам интерполирует между командами (sendcmd -> crop x = X).
```

Практические замечания. Во-первых, окно кропа не должно выходить за границы кадра: x_expr ограничиваем через clip(x, 0, in_w-608) — sendcmd записывает уже безопасные значения, но ограничитель оставляем как страховку. Во-вторых, при двух говорящих (допрос, спор сторон) переключение окна делаем с анимацией ~0.4 с — EMA это даёт бесплатно. В-третьих, если детекция не нашла лиц (запись с плана зала сверху), корректный fallback — текущий центральный кроп: фича обязана деградировать, а не ломать рендер. Оценка ОЗУ: MediaPipe +150 МБ в фазе анализа, в пик рендера не попадает; трек считается за ~0.3× реального времени на CPU ноутбука.

5. КФ-3. Цитаты, авто-зум и звуковые акценты

Submagic собирает «динамику» клипа из трёх приёмов: авто-зум (fast zoom, crash zoom, smooth zoom) на акцентных словах, плашки-подложки под ключевые фразы и короткие SFX (whoosh, click) на переходах. Для судебного контента адаптируем это в «цитатный» формат: LLM выбирает 1–2 самых сильных заявления спикера (глава 6 даёт их из транскрипта), на этих секундах рендер делает zoom-punch до 112% и поднимает цитатную плашку — цитата дублирует речь текстом и даёт зрителю «якорь», которым можно поделиться.

FFmpeg: цитата + zoom-punch + SFX в одном filtergraph

```
# Цитатная плашка (lower third) с fade-in/out + zoom-punch на 1.2-4.8 сек
ffmpeg -i cut.mp4 -i whoosh.mp3 -filter_complex "
 [0:v]drawtext=fontfile=Montserrat-ExtraBold.ttf:
       text='«Я не подписывал этот протокол»':
       fontcolor=white:fontsize=44:borderw=3:bordercolor=0x1A4A7A:
       box=1:boxcolor=0x142840@0.72:boxborderw=18:
       x=(w-text_w)/2:y=h*0.78:
       alpha='if(lt(t,1.2),0,if(lt(t,1.7),(t-1.2)/0.5,
             if(lt(t,4.3),1,if(lt(t,4.8),(4.8-t)/0.5,0))))':
       enable='between(t,1.2,4.8)'[txt];
 [txt]scale=1080:1920:force_original_aspect_ratio=increase,
      crop=1080:1920:
      x='(iw-ow/1.12)/2*if(between(t,1.2,4.8),1,0)+iw/2*if(...,0,1)':
      y='(ih-oh/1.12)/2*if(between(t,1.2,4.8),1,0)+ih/2*if(...,0,1)'[v];
 [1:a]adelay=1150|1150,apad[sfx];
 [v][sfx]amix=inputs=2:duration=first:weights=1 0.35[a]
" -map "[v]" -map "[a]" -c:v libx264 -crf 19 -c:a aac quote_out.mp4
```

Проще и надёжнее считать зум через отдельный проход: сначала рендерим клип с плашкой (drawtext), затем для зум-сегментов применяем zoompan (z="if(lte(on,120),1.12,1.0)") или предвычисленный crop — это легче отлаживать, чем один гигантский filtergraph. SFX обязаны быть приглушёнными (weights 0.3–0.4) и одинаковыми для всех клипов канала — так формируется узнаваемый звуковой почерк «Монтажной Кухни». Все элементы включаются переключателями в params задачи: quotes (bool), zoom_punch (bool), sfx (bool), чтобы владелец мог выключить всё одним тапом для строгих сюжетов.

Ритм смены визуальной активности — ещё один урок CapCut: изменение в кадре каждые 3–5 секунд удерживает внимание. У нас это бесплатно дают: караоке-слова (0.3–0.8 с), цитатные плашки (4–6 с), зум-панчи (0.5 с) и уже существующая нарезка пауз. Итого плотность событий кадра сопоставима с вирусными клипами, без единой вставки чужеродного контента.

6. КФ-4. Вирусный хук: заголовки и скоринг виральности

Главный урок Opus Clip: 40% автоматически нарезанных клипов создатели всё равно отбрасывают — значит, ценность не в автопилоте, а в скоринге и контроле. Второй урок: первые 3 секунды решают всё — клип должен начинаться с кульминационной фразы (cold open), а не с начала ответа. Оба решения реализуются чистым LLM-слоем поверх транскрипта, который у нас уже есть после Whisper: никакого видеотракта, ноль риска для рендера.

Схема: после транскрипции воркер передаёт сегменты с таймкодами в LLM (дешёвая модель, JSON-режим) с промптом на три задачи: (1) выделить 2–3 кандидата-хука — фразы-кульминации с их таймкодами, чтобы клип начинался за 0.5 с до сильного заявления; (2) сгенерировать 5 вариантов заголовка на укр + 5 на рус (шаблоны: вопрос, число-факт, конфликт, интрига-недосказанность, прямое заявление); (3) вернуть описания и хэштеги для YouTube/Shorts. Скоринг виральности считаем двумя слоями: эвристика (детерминированная, быстро) + LLM-балл (1–10). Итоговый балл показываем в Mini App и в боте рядом с очередью:

Скоринг виральности + промпт LLM-редактора

```
# Эвристический скоринг (детерминированный, без LLM) — 0..100
score = 0
score += 22 if re.search(r'\?', hook_text) else 0          # вопрос в хуке
score += 15 if re.search(r'\d', hook_text) else 0           # цифры/факты
score += 18 * (hook_words <= 9)                              # хук короче 9 слов
score += 20 if 21 <= dur <= 34 else 8                        # sweet-spot длит-ти
score += 15 if wpm in range(140, 190) else 6                 # темп речи
score += 10 * has_conflict_lexicon(hook_text)                # лексика конфликта
final = round(0.55 * score + 0.45 * llm_score * 10)

# Промпт LLM (JSON-mode, температура 0.7):
# "Ты редактор вертикальных шортсов о судебных заседаниях. Дан транскрипт
#  с таймкодами. Верни JSON: {hooks:[{text,t_start,t_end,score}],
#  titles_uk:[5], titles_ru:[5], description, hashtags:[...]}.
#  Правила: хук = кульминация, не начало ответа; заголовки <= 60 знаков;
#  без кликбейта, противоречащего фактам записи.
```

Интеграция в продукт: балл и заголовки пишутся в Job.params/Event ещё на стадии транскрипции; в Mini App (вкладка «Задачи») показываем бейдж «Виральность 78» и список заголовков — владелец выбирает до публикации; бот присылает топ-заголовок вместе с готовым роликом. Для витрины тот же балл сортирует превью. Это ровно та роль, которую у Opus Clip играет Virality Score, но с честной двухслойной формулой, которую мы можем объяснить и настраивать под свой канал.

> *«Хук — это не обман зрителя, а честная кульминация, вынесенная в начало. Судебное заседание само пишет себе заголовки — наша задача лишь вовремя их заметить и измерить.»*

7. Дорожная карта внедрения в video-montage

Все четыре фичи ложатся в существующий FIFO-конвейер как новые стадии между транскрипцией и финальным рендером; последовательный рендер и бюджет ОЗУ сохраняются. Порядок — по цене входа и эффекту: караоке (неделя 1), LLM-хуки (неделя 2), цитаты с зумом (недели 3–4), спикер-кроп (недели 5–6). Каждая неделя заканчивается боевым выпуском клипа на реальном заседании — это обязательный чек-пойнт приёмки, а не внутренний тест.

Неделя	Стадия конвейера	Что делаем	Чек-пойнт
1	render: subtitles	karaoke_subs.py: word-timestamps → ASS \k; слои укр+рус; выбор акцента в params	Клип с караоке опубликован; сравнить удержание со старым
2	transcribe → analyze	LLM-слой: хуки/заголовки/скоринг; бейдж «Виральность» в Mini App и боте	Топ-1 заголовок использован в публикации
3–4	render: overlay	drawtext-цитаты + zoom-punch + SFX; переключатели в форме Shorts	A/B: клип с цитатой против без
5–6	pre-render: reframe	MediaPipe-трек + sendcmd-кроп; fallback на центр; прогон на 3 заседаниях	Спикер в кадре ≥95% времени клипа

Таблица 3. Шестинедельная дорожная карта (по одной боевой публикации в неделю)

Бюджет ОЗУ остаётся в безопасной зоне: караоке и цитаты исполняются внутри существующего ffmpeg-процесса (+0 ГБ), LLM-слой — это HTTP-запрос (+0.2 ГБ на кеш ответа), MediaPipe добавляет ~0.15 ГБ только в фазе анализа кадров, которая идёт до пиковой фазы рендера сегментов. Пиковая сумма по стадиям не превышает текущие 7.6 ГБ, запас до 12 ГБ сохраняется, FIFO с mem_limit трогать не нужно.

Метрики успеха на 30 дней после внедрения КФ-1 и КФ-4: средний процент досмотра Shorts растёт с базового уровня минимум на 15 п.п., CTR обложек с вирусными заголовками — на 10%, доля клипов, опубликованных без ручной правки заголовка, ≥60%. Если метрики не движутся — в первую очередь проверяем качество хуков (не переставили ли мы кульминацию в скучное место) и читаемость караоке на телефоне (76 px минимум). Следующий горизонт после шести недель — выборка b-roll из архивов самого канала (аналог Magic B-roll, но только собственные кадры) и автоматическая генерация обложек из хук-кадра с плашкой-заголовком.